SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Modal

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems.. (BL-6)

Assessment Tool Weightage: 30%

QUESTIONS: **Total Marks:25**

Q1. Transfer Learning

Design a transfer learning framework for a plant disease detection system using a pre-trained CNN model. Justify the choice of layers to freeze and fine-tune, and propose suitable data augmentation techniques to improve classification performance.

A) A **transfer learning framework** can be developed using a pre-trained CNN such as **ResNet50, VGG16, or MobileNetV2**. The model is first trained on a large dataset such as ImageNet and then adapted to classify plant diseases.

1. Framework

Input Leaf Image → Preprocessing → Pre-trained CNN → Fine-tuning → Classification Layer → Disease Prediction

Example classes:

- Healthy leaf
- Leaf spot
- Rust
- Powdery mildew
- Bacterial blight

2. Choice of Layers

Freeze initial layers:

- Freeze the first few convolutional layers because they learn general features such as **edges, colours, textures, and shapes**.
- This reduces training time and prevents overfitting when the plant dataset is small.

Fine-tune deeper layers:

- Unfreeze the last few convolutional layers because they learn more **task-specific features**, such as spots, lesions, discoloration, and leaf patterns.
- Replace the original classification layer with a new **Dense/SoftMax layer** corresponding to the number of plant disease classes.

3. Data Augmentation

To increase the diversity of training images and improve generalization:

Technique	**Purpose**
Rotation	Handles different leaf orientations
Horizontal/Vertical Flip	Increases image variation
Zoom	Handles different distances
Width/Height Shift	Handles different leaf positions
Brightness adjustment	Handles different lighting conditions
Shearing	Handles changes in viewing angle
Random cropping	Focuses on different leaf regions

4. Training Process

1. Load the pre-trained CNN.
2. Remove its original classification layer.
3. Freeze most convolutional layers.
4. Add a new classification layer.
5. Apply data augmentation to training images.
6. Train the new classification layer.
7. Unfreeze the final few CNN layers.
8. Fine-tune them using a **small learning rate**.
9. Evaluate using accuracy, precision, recall, and F1-score.

5. Result

The system can identify the disease from a leaf image by learning both **general visual features from the pre-trained CNN** and **plant-disease-specific features through fine-tuning**. This generally requires less training data and time than training a CNN completely from scratch.

Q2. DenseNet and PixelNet

Develop a CNN-based architecture by integrating DenseNet and PixelNet concepts for semantic image segmentation in autonomous driving. Explain how dense connectivity and pixel-wise prediction improve the overall segmentation accuracy.

A) A CNN architecture can combine **DenseNet's dense connectivity** with **PixelNet's pixelwise prediction** to segment objects such as roads, cars, pedestrians, buildings, and traffic signs in autonomous driving.

1. Architecture

Input Image → DenseNet Feature Extractor → Feature Fusion → Pixel-wise Prediction → Segmentation Map

- **Input:** Road scene image, e.g., 512 × 512 pixels.
- **DenseNet:** Extracts detailed features from the image.
- **Feature Fusion:** Combines low-level and high-level features.
- **Pixel-wise Prediction:** Predicts a class for every pixel.
- **Output:** Segmentation map showing the class of each pixel.

2. Dense Connectivity

In DenseNet, each layer receives feature maps from **all previous layers**:

$F_1 \rightarrow F_2 \rightarrow F_3 \rightarrow F_4$

Each layer can reuse previously extracted features.

Advantages:

- Improves feature reuse.
- Preserves important low-level information.
- Reduces the vanishing-gradient problem.
- Helps detect small objects such as pedestrians and traffic signs.
- Requires fewer parameters compared with some traditional CNNs.

3. Pixel-wise Prediction

PixelNet focuses on predicting a class for **each individual pixel**.

For example:

Pixel Region Predicted Class

Road pixels Road

Vehicle pixels Car

Human pixels Pedestrian

Pixel Region Predicted Class

Sign pixels Traffic Sign

Sky pixels Sky

A **Softmax layer** produces class probabilities for every pixel.

4. Combined Architecture

1. The input image is given to DenseNet.
2. Dense connections extract and reuse visual features.
3. Features from different levels are fused to retain both **spatial details and semantic information**.
4. A decoder/upsampling stage restores the original image resolution.
5. PixelNet-style prediction assigns a class to every pixel.
6. The final output is a complete semantic segmentation map.

5. How Accuracy Improves

DenseNet improves feature extraction, especially for boundaries and small objects, while **PixelNet provides precise pixel-level classification**.

Therefore:

Dense Connectivity → Better Feature Reuse → Richer Features → Pixel-wise Prediction → Accurate Segmentation

This combination is useful in autonomous driving because accurate segmentation helps the vehicle distinguish **roads, vehicles, pedestrians, obstacles, and traffic signs**, supporting safer navigation.

Q3. Bidirectional RNN and Encoder–Decoder

Create an encoder–decoder sequence-to-sequence model using Bidirectional RNNs for multilingual machine translation. Illustrate the architecture and explain how bidirectional context enhances translation quality.

A)**1.Introduction:**
A Bidirectional RNN-based Encoder–Decoder model can be used for multilingual machine translation. It converts a sentence from a source language such as English into a target language such as Tamil, Hindi, or French.

2.Encoder:
The encoder takes the input sentence and converts each word into a numerical vector using word embeddings. A Bidirectional RNN processes the sentence in two directions: from left to

right and from right to left. The information from both directions is combined to create a meaningful representation of the complete sentence.

3.Decoder:
The decoder receives the information generated by the encoder and produces the translated sentence one word at a time. At each step, it uses the previously generated word and the encoded information to predict the next word.

4.BidirectionalContext:
A normal RNN mainly considers previous words, whereas a Bidirectional RNN considers both previous and future words. This helps the model understand the meaning of words based on their complete context and reduces ambiguity during translation.

5.MultilingualTranslation:
The same architecture can be trained with multiple language pairs. The encoder learns the meaning of sentences independent of language, while the decoder generates the sentence in the required target language.

6.Advantages:
Bidirectional context improves sentence understanding, handles different word orders, resolves ambiguous words, and improves translation accuracy. Therefore, the combination of **BiRNN + Encoder–Decoder** provides an effective architecture for multilingual machine translation.

Q4. BPTT and LSTM

Design an LSTM-based network to predict stock market trends from historical time-series data. Explain how Backpropagation Through Time (BPTT) is applied during training and justify why LSTM is preferred over a standard RNN.

A) **1. Introduction**

An **LSTM-based network** can predict stock market trends using historical time-series data. The input may contain opening price, closing price, high, low, and trading volume. The model learns patterns from previous days and predicts whether the stock price will increase or decrease.

2. Architecture

Historical Stock Data → Preprocessing → LSTM Layer → Dense Layer → Output

- Historical stock data is collected and normalized.
- A sequence of previous time steps is given to the LSTM.
- The LSTM learns temporal patterns.
- A Dense layer produces the final prediction.
- **Sigmoid** can be used for predicting *Up/Down* trends.

3. BPTT Training

Backpropagation Through Time (BPTT) is used to train the LSTM. The sequence is processed step-by-step during the forward pass. The predicted output is compared with the

actual stock trend to calculate the loss. The error is then propagated backward through the different time steps, and the network weights are updated using an optimizer such as Adam.

4. Why LSTM is Preferred

A standard RNN may have difficulty remembering information from long sequences because of the **vanishing-gradient problem**. LSTM solves this using a **cell state and three gates: input gate, forget gate, and output gate**. These gates control which information should be stored, removed, or passed to the next time step.

5. Advantages

- Learns long-term dependencies.
- Handles sequential stock data effectively.
- Reduces the vanishing-gradient problem.
- Captures important historical patterns.
- Provides better learning for long time-series sequences.

6. Conclusion

Thus, an **LSTM with BPTT** is suitable for stock market trend prediction because LSTM can remember important information over long periods, while BPTT helps train the network by propagating errors through the sequence.

Q5. Integrated Deep Learning Model

Develop a deep learning solution for automatic video caption generation by integrating transfer learning for feature extraction with an LSTM-based encoder–decoder architecture. Justify the role of each component and explain how the complete system generates meaningful captions.

A) **1. Introduction**

An automatic video captioning system generates a meaningful text description for a given video. The system can combine **transfer learning for visual feature extraction** with an **LSTM-based encoder–decoder** to understand video content and generate captions.

2. Architecture

Input Video → Frame Extraction → Pre-trained CNN → Feature Extraction → LSTM Encoder → LSTM Decoder → Caption

3. Transfer Learning for Feature Extraction

A pre-trained CNN such as **ResNet50 or InceptionV3** is used to extract visual features from video frames. The CNN has already learned general features such as objects, shapes, and textures from a large image dataset. Most CNN layers are frozen, and the final classification layer is removed. This reduces training time and allows the model to focus on learning videospecific information.

4. LSTM Encoder

The extracted features from consecutive video frames are given to an **LSTM encoder**. The LSTM processes the features as a sequence and learns the temporal information in the video. For example, it can understand the sequence **person → picks up → ball → throws ball**.

5. LSTM Decoder

The decoder receives the encoded video information and generates the caption **word by word**. It starts with a start token and predicts the next word based on the video features and previously generated words.

Example:
Generated Caption: *"A person is playing with a dog."*

6. Role of Each Component

Component	Role
Video Frames	Provide visual information
Pre-trained CNN	Extracts important visual features
Transfer Learning	Reuses knowledge from large datasets
LSTM Encoder	Learns temporal relationships between frames
LSTM Decoder	Generates words sequentially
Output Layer	Predicts the next word

7. Caption Generation Process

First, the video is divided into frames. Each frame is passed through the pre-trained CNN to obtain feature vectors. These features are given to the LSTM encoder, which learns the sequence of events. The decoder then uses this encoded information to predict words one by one until an end token is generated.

8. Conclusion

The integrated model combines the **strong visual feature extraction capability of transfer learning** with the **sequence-learning ability of LSTM**. This allows the system to understand both **what is happening in the video and the order in which events occur**, resulting in meaningful and context-aware captions.

Code of Conduct:

I J.Raju (192325117) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J.Raju

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases